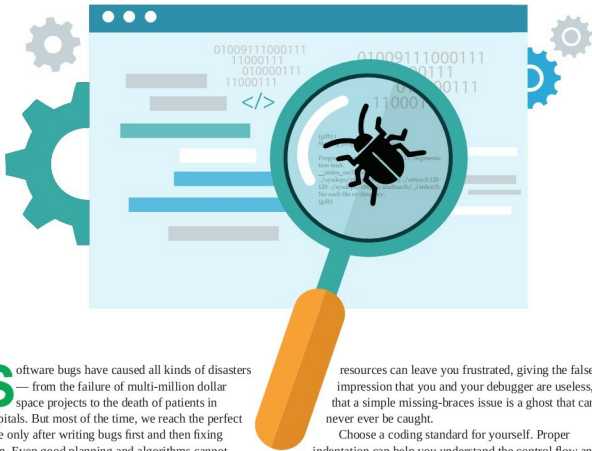


Debugging Your Programs: From *printf* to GDB

The essence of writing code is to also be able to debug it. Debugging programs not only help to remove kinks from the code, but also hone the skills of the code writer. This article discusses various debugging tools and techniques to enable code writers to improve their skills.



Software bugs have caused all kinds of disasters — from the failure of multi-million dollar space projects to the death of patients in hospitals. But most of the time, we reach the perfect code only after writing bugs first and then fixing them. Even good planning and algorithms cannot foresee some runtime misfortunes. The only thing we can do is to get armed with good debugging skills.

There are user-friendly IDEs with simple *Play* buttons to help you start debugging, and Web browsers have made such features a part of our day-to-day browsing experience. But if you have skipped the basics and moved straight to these fancy tools, give the classic tools like *printf* and GDB a try, and you will feel you have spent your time well.

Writing code that makes sense

There might be a hundred tools to help you debug your program, from *printf* to GDB, or some fancy graphical stuff. But a productive debugging session requires your code to be readable. A code that doesn't reveal the hierarchy, control flow, or the meaning of variables, functions and other

resources can leave you frustrated, giving the false impression that you and your debugger are useless, or that a simple missing-braces issue is a ghost that can never ever be caught.

Choose a coding standard for yourself. Proper indentation can help you understand the control flow and scope. It has often been observed that adding indentation to an erroneous program reveals the mistakes automatically, even causing you to fix them unconsciously. If you'd like to get automatic indentation for the C code that you have already written, you can use `GNU indent`, an FSF implementation of the UNIX `indent` command.

Names should be as meaningful as possible and even the letter case you choose should matter. For example, you can choose full-upper-case to name constants.

Finally, split your code into multiple functions and files, with proper custom header files, as needed.

printf debugging

As its name suggests, this is the practice of inserting output statements in usually unwanted places to monitor